

IES FRANCISCO DE GOYA – LA ELIPA DEPARTAMENTO DE INFORMÁTICA				Dirección del Área Territorial de Madrid Capital CONSEJERÍA DE EDUCACIÓN, JUVENTUD Y DEPORTE Comunidad de Madrid 
Titulación:	DAM/DAW	Curso:	2025/2026	
Módulo:	Bases de Datos	Fecha:	Mayo 2026	
Examen:	UT7: Variante 1 - Gestión de Alquileres	Duración:	60 min	
Nombre:		Calificación:		
Apellidos:				
RA:	RA5: Desarrolla procedimientos almacenados evaluando y utilizando las sentencias del lenguaje incorporado en el sistema gestor de bases de datos.			

Instrucciones

- Los ejercicios deben resolverse utilizando sintaxis estándar de MySQL.
- Se valorará la limpieza del código, el uso de delimitadores y la correcta definición de tipos.
- La base de datos de referencia para todos los ejercicios es **Sakila**.

Ejercicio 1 (4 puntos) – (RA5-CE.g) Procedimiento con bucle

Crea un procedimiento almacenado llamado `p_duplicate_inventory` que genere copias de seguridad en la tabla `inventory`.

- Recibe: `p_film_id`, `p_store_id`, `p_num_copies` (IN) y `p_last_id` (OUT).
- Debe usar un bucle **WHILE** para insertar exactamente el número de copias solicitado.
- Valida con un **IF** que el número de copias sea positivo.
- Al finalizar, el parámetro OUT `p_last_id` debe contener el ID del último registro insertado usando `LAST_INSERT_ID()`.

Ejemplo de ejecución:

```
CALL p_duplicate_inventory(1, 1, 3, @ultimo_id);
SELECT @ultimo_id;
```

Resultado esperado: El SELECT mostrará el ID del último registro insertado en `inventory` (ej. **4584** asumiendo que partimos del inventario original de Sakila).

Pista: Para obtener el ID generado por el último INSERT en una columna autoincremental, utiliza la función de sistema `LAST_INSERT_ID()`.

Ejercicio 2 (3 puntos) Trigger de Seguridad

Implementa un trigger llamado `tr_check_store_manager` en la tabla `store` que actúe antes de actualizar un registro.

- Evita que un empleado sea manager de dos tiendas distintas (2 pts).
- En el mensaje de error de SIGNAL, incluye el nombre del usuario del sistema que intenta el cambio usando la función `USER()`. (1 pto).

Ejemplo de ejecución:

```
UPDATE store SET manager_staff_id = 1 WHERE store_id = 2;
```

Resultado esperado: ERROR 1644 (45000): Error: Manager duplicado. Usuario: admin@localhost

Nota: La función `USER()` no se puede utilizar directamente en `SIGNAL SET MESSAGE_TEXT` porque no acepta expresiones. Debes crear una variable (ej. `v_error_msg`) donde concatenar el mensaje con `USER()`, y luego usar esa variable en `SIGNAL`.

Ejercicio 3 (3 puntos) Función No Determinista

Crea una función almacenada llamada `f_calculate_late_fee` que calcule la penalización por retraso en la devolución de películas.

- Recibe el ID de un alquiler (`p_rental_id`) y devuelve un `DECIMAL(10,2)`.
- Si el alquiler no se ha devuelto (`return_date` es NULL), usa la fecha actual (`NOW()`) para el cálculo.
- Calcula los días de retraso respecto a la fecha esperada (7 días después del alquiler).
- Devuelve 1.50 euros por cada día de retraso.

Ejemplo de ejecución:

```
SELECT f_calculate_late_fee(4);
```

Resultado esperado: Un valor de **4.50** (el alquiler 4 se devolvió 3 días después de la fecha debida: 2005-06-03 frente a 2005-05-31).

Nota: No olvides marcar la función como `NOT DETERMINISTIC` ya que utiliza `NOW()`. También debes especificar `READS SQL DATA` porque lee de la tabla `rental`.